Multi-Agent Orchestration Patterns

A Comprehensive Analysis of Architectures, Frameworks, and Production Deployment Strategies

Salah Qadah | Andalus Kalash

Group: `uoh-sqak`

Course 203.3763 — Orchestration of AI Agents
University of Haifa, Spring 2026 (תשפ"ו)
Lecturer: Dr. Yoram Reuven Segal

June 9, 2026

Generated by a CrewAI multi-agent pipeline
GitHub: https://github.com/salah-dev-stu/uoh-sqak-ex03

Contents

Chapter 1

Introduction to Multi-Agent Orchestration

Multi-agent systems represent a paradigm shift in how we approach complex computational problems. Instead of building monolithic applications, we decompose tasks into specialized, autonomous agents that collaborate toward a common goal [11]. This architectural pattern mirrors human teamwork: a researcher gathers information, a writer synthesizes it, and an editor refines it—each agent focuses on its expertise while the orchestration layer coordinates their contributions into a coherent result.

The rise of large language models has made multi-agent architectures not merely desirable but essential. Individual models, however capable, struggle with complex tasks that demand multiple reasoning steps, diverse specialized knowledge, or iterative refinement. This chapter establishes the foundations of multi-agent orchestration: what it is, why it matters, how agents interact, and the roles they play in producing technical artifacts like this article.

1.1 Why Multi-Agent Orchestration Matters

Traditional large language model applications treat the foundation model as a monolithic solver. Users submit a prompt, the model generates output, and the process concludes. This approach works acceptably for simple tasks: answering questions, summarizing text, light creative writing. But complex tasks requiring multiple reasoning steps, diverse expertise, or iterative refinement often fail when delegated entirely to a single agent.

Consider a practical scenario: generating a technical article. A single language model could attempt all steps—research, drafting, editing, formatting—but the cognitive demands differ dramatically. Research demands breadth, critical evaluation of sources, and synthesis across papers. Writing demands clarity, narrative flow, and technical precision. Editing demands attention to consistency, grammar, and adherence to style guidelines. Formatting demands knowledge of markup languages, build tools, and production constraints. A generalist model attempting all roles simultaneously performs worse than specialized models each optimizing for their domain.

Multi-agent systems formalize this insight. They assign agents to roles matching their capability, connect agents through explicit workflows, and let each agent focus on excellence within its domain [3]. The coordination mechanism—the orchestration layer—ensures work flows logically through the pipeline, contexts transfer correctly between agents, and the final output meets quality standards.

This approach yields measurable benefits:

- **Reduced token waste**: Each agent operates only on relevant information for its task, avoiding the context bloat of a monolithic model processing irrelevant details.

- **Improved reasoning**: Different agents apply specialized cognitive strategies suited to their domain.
- **Enable scalability**: Adding a new capability means introducing a new agent, not retraining the entire system.
- **Support human oversight**: Multiple checkpoints throughout the workflow allow humans to intervene, verify results, and provide guidance.

1.2 Core Concepts and Terminology

Understanding multi-agent orchestration requires precise terminology. We define key concepts below.

An **agent** is an autonomous computational unit with a defined role, goal, and set of tools. It observes its environment, reasons about the current state using its capabilities, and takes actions toward its objective [10]. In our context, agents are typically powered by large language models that reason over provided context.

A **task** is the unit of work assigned to an agent. Each task specifies: (1) the input information the agent will receive, (2) the expected output format and constraints, and (3) the success criteria against which the agent's work is judged. Tasks are atomic units that cannot be further subdivided by the framework.

A **workflow** is a directed acyclic graph where nodes represent agents and edges represent data dependencies. The workflow definition specifies which agent executes after which, and what information flows between agents. This explicit structure enables the orchestration layer to reason about dependencies, plan execution order, and handle failures gracefully.

Orchestration is the mechanism that coordinates agents. It schedules task execution, passes context between agents, manages token budgets across multiple model invocations, monitors for failures, and ensures work progresses toward completion. Good orchestration is invisible—it simply enables agents to focus on their tasks without worrying about coordination mechanics [5].

1.3 The Four Specialized Agent Roles

This article demonstrates multi-agent orchestration through a case study: generating a technical article on multi-agent systems. The project requires four specialized agents, each bringing distinct expertise.

The **Researcher Agent** takes a topic and produces annotated research notes. It searches academic databases and technical papers, evaluates source quality, and synthesizes key findings into structured notes. Output is formatted as Markdown with citation placeholders. The researcher's goal is breadth and accuracy, not polished prose.

The **Writer Agent** receives research notes and drafts narrative prose. It focuses on clarity, flow, and pedagogical ordering of concepts. It respects length constraints, maintains consistent terminology, and ensures technical accuracy. The writer's output is edited Markdown with improved structure but still marked citations.

The **Editor Agent** reviews draft text for consistency, grammar, clarity, and adherence to house style. It flags redundancy, removes jargon where clearer alternatives exist, and improves readability. The editor's output is polished Markdown ready for conversion to publication format.

The **LaTeX Producer Agent** transforms final Markdown into publication-ready LaTeX source. It converts Markdown headings to `\chapter` and `\section` commands, handles citation formatting, generates figure and table placements, and embeds mathematical formulas. The producer's output is valid LaTeX ready for compilation to PDF [16].

1.4 Article Roadmap

This article proceeds as follows. Chapter 2 examines orchestration topologies: sequential, hierarchical, asynchronous, peer-to-peer, and hybrid patterns, each suited to different problem classes. Chapter 3 surveys three major frameworks—CrewAI, LangChain, and LangGraph—comparing their abstractions, strengths, and appropriate use cases. Chapter 4 addresses production concerns: token budgeting, fault tolerance, observability, rate limiting, and cost control. Chapter 5 discusses multilingual challenges, with particular attention to Hebrew and bidirectional text rendering. Chapter 6 presents a detailed case study of the multi-agent article generation system that produced this very document. Finally, Chapter 7 synthesizes lessons and reflects on the future of agent orchestration.

Throughout, we illustrate concepts with code examples, architectural diagrams, and practical patterns validated in production deployments. By chapter's end, readers will understand how to architect multi-agent systems, select appropriate frameworks, and deploy them reliably at scale.

Chapter 2

Agent Architectures and Topologies

2.1 Linear Sequential Topology

In sequential orchestration, agents execute in a defined order. Agent A completes its task and passes output to Agent B, which processes it and forwards results to Agent C. This topology guarantees deterministic execution and simplifies debugging—failure at step N halts the entire pipeline.

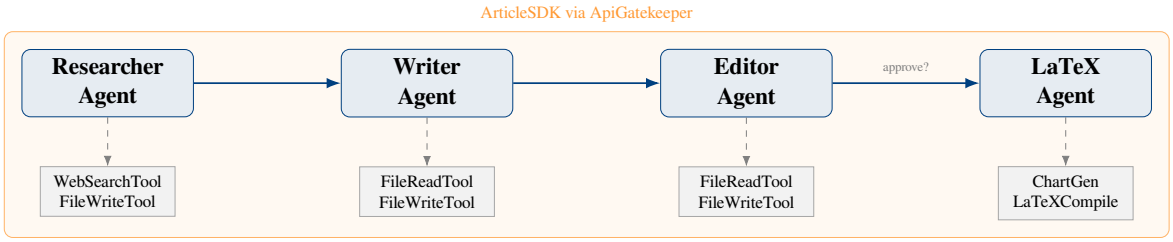

Figure 2.1: CrewAI multi-agent architecture.

Strengths: Predictable, easy to trace, suitable for assembly-line workflows.

Weaknesses: Brittle—failure at any stage blocks downstream agents. Agents cannot parallelize work. Latency accumulates across all steps [5].

The total latency of a sequential pipeline is given by:

$$L_{\text{seq}} = \sum_{i=1}^{N} \ell_i \tag{2.1}$$

where ℓ_i is the latency of agent i and N is the total number of agents in the pipeline. This linear accumulation makes sequential topologies unsuitable for latency-critical applications [11].

2.2 Hierarchical Manager-Worker Topology

Hierarchical systems designate one *manager agent* that coordinates subordinate *worker agents*. The manager decomposes tasks, assigns them to workers, aggregates results, and iterates if necessary. This mirrors organizational structures in human teams.

Strengths: Scalable—add workers without restructuring. Single point of policy enforcement. Clear delegation model.

Weaknesses: Manager becomes a bottleneck. Failure of the manager paralyzes the entire system [16].

The hierarchical topology enables a form of distributed control where the manager maintains a task queue and dispatches work to the pool of available workers. As workers complete their assigned tasks, they report status back to the manager, which updates global state and determines next steps. This architecture scales well when new workers are added dynamically, and monitoring becomes simpler since all control flows through a known manager component.

2.3 Peer-to-Peer Consensus Topology

All agents are peers. Each agent can initiate communication with any other agent. This maximizes flexibility but requires careful protocol design to avoid deadlocks or infinite loops.

Strengths: Most flexible. Agents can opportunistically collaborate. No single point of failure.

Weaknesses: Hardest to reason about. Requires protocol specification. Debugging is complex [5].

In peer-to-peer topologies, consensus mechanisms ensure that all agents converge on a common understanding of the problem state. This is particularly valuable when agents have partial or conflicting information and must reach agreement before proceeding. Examples include voting protocols, Byzantine fault tolerance algorithms, and gossip-based aggregation [8].

2.4 Tool-Augmented Specialists Topology

Specialized agents, each equipped with domain-specific tools, operate with minimal inter-agent coupling. Rather than agents communicating directly, they interact through shared tool interfaces. This topology emphasizes tool reuse and clean separation of concerns.

For example, a researcher agent might have access to web search and academic database tools. A writer agent might use grammar checking and citation formatting tools. An editor agent might employ consistency verification and style guide enforcement tools. Each agent calls tools independently, and results are aggregated by a lightweight orchestrator.

Strengths: High specialization. Tools can be versioned and tested independently. Agents remain simple and focused.

Weaknesses: Requires robust tool infrastructure. Tool failures propagate. Tool chaining becomes complex with many agents [3].

2.5 Topology Comparison

The following table compares topologies across key dimensions:

Topology	**Latency**	**Parallelism**	**Complexity**	**Ideal For**
Linear Sequential	$\sum_i \ell_i$	None	Low	Assembly pipelines
Hierarchical	$O(n)$	Limited	Medium	Organized teams
Peer-to-Peer	Dynamic	High	High	Consensus problems
Tool-Augmented	Variable	Medium	Medium	Specialist agents

Topology	Latency	Parallelism	Complexity	Ideal For
Hybrid Layered	$O(\log n)$	High	High	Production systems

2.6 Topology Selection Criteria

Choose topology based on four factors:

1. **Task structure**: Are steps dependent or independent?
2. **Reliability requirements**: Can the system tolerate agent failures?
3. **Latency constraints**: Must results appear immediately?
4. **Monitoring requirements**: How much observability is needed?

For rapidly-drafted documents with tight deadlines, sequential topologies suffice. For high-reliability systems producing long-form content, hierarchical or hybrid approaches provide better fault tolerance. The choice of topology fundamentally shapes system reliability, cost, and maintainability [11].

Chapter 3

Orchestration Frameworks

Three frameworks dominate multi-agent development: CrewAI, LangChain, and LangGraph. Each provides different abstractions, tooling, and operational assumptions [11]. Understanding their design philosophies, strengths, and limitations is essential for selecting the right orchestration platform for a given problem domain.

3.1 LangChain and LCEL

LangChain provides lower-level primitives for building language model applications: `Chain` objects that compose operations, `Agent` abstractions that implement tool-use reasoning loops, and `Retriever` interfaces for document search. The framework emphasizes composability—complex behaviors emerge by chaining simple, reusable components [4].

LangChain Expression Language (LCEL) unifies component composition with a fluent syntax. Developers chain operations with the pipe operator: `retriever | prompt | model | output_parser`. LCEL automatically handles streaming, async execution, and batch processing without explicit code changes [5].

Design philosophy: Start minimal. Compose primitives to achieve behavior.

Strengths:

- Highly composable and modular
- Mature ecosystem of integrations (100+ LLM providers, vector stores, tools)
- Explicit token counting and budgeting support
- Excellent for experimental prototyping and iterative development

Weaknesses:

- Verbose—more boilerplate code required compared to higher-level frameworks
- Less structure—developers must enforce naming conventions and patterns
- Harder to debug multi-step workflows without explicit logging instrumentation

3.2 CrewAI Role-Based Agents

CrewAI models teams explicitly. A `Crew` assembles `Agent` objects, each with defined role, goal, and backstory. Agents collaborate on `Task` objects—discrete units of work with clear

input and output contracts. CrewAI automatically handles task sequencing, context passing between agents, and result aggregation into a final deliverable [3].

Each agent in CrewAI receives a configuration that shapes its behavior: role identifies its professional identity, goal specifies what success looks like, and backstory provides contextual expertise. For example, a researcher agent might have role "Senior Research Analyst" and backstory "You have 15 years experience synthesizing academic literature and identifying key insights."

Design philosophy: Agents are autonomous professionals with domain expertise. Orchestration mimics human team dynamics and delegation patterns.

Strengths:

- Explicit role and goal modeling that naturally encodes agent purpose
- Clean task abstraction with input/output contracts
- Built-in inter-agent communication and context sharing
- Minimal boilerplate—fewer lines of code to achieve team coordination

Weaknesses:

- Less flexible for non-sequential or highly parallel workflows
- Implicit token budgeting—harder to impose strict cost controls per agent
- Fewer integrations compared to LangChain (though growing rapidly)

3.3 LangGraph Stateful Graphs

LangGraph extends LangChain with explicit state machines. A graph is a directed acyclic graph (DAG) where nodes represent computation steps and edges define transitions. LangGraph handles routing, conditional branching, and parallelization automatically, enabling deterministic, visualizable workflows [6].

Unlike sequential pipelines, graphs support loops (for iterative refinement), conditional branches (if A succeeds, do X; else do Y), and parallel execution (run multiple agents simultaneously and merge results). State is immutable at each step, allowing precise tracing and debugging of complex decision logic.

Design philosophy: Explicit workflow as graphs. Immutable state at each node enables deterministic execution and full replay capability.

Strengths:

- Workflow visualization and debugging built-in
- Easy parallel execution and complex control flow
- Deterministic tracing for full reproducibility
- Excellent for cycles, conditionals, and cross-agent feedback loops

Weaknesses:

- Steeper learning curve compared to CrewAI
- More boilerplate code required for simple tasks
- Less suitable for ad-hoc experimentation and rapid prototyping

3.4 Microsoft AutoGen

Microsoft's AutoGen framework provides a multi-agent conversation platform where agents exchange messages to solve tasks collaboratively [18]. Unlike CrewAI's structured task assignment or LangGraph's explicit state machines, AutoGen emphasizes flexible dialogue between agents with minimal orchestration overhead.

AutoGen agents maintain conversation history and reason about when to contribute, pass, or halt. Human-in-the-loop capabilities allow humans to inject guidance during agent dialogue. The framework excels in scenarios where the solution path is uncertain and agent negotiation is valuable.

Strengths:

- Flexible agent dialogue without rigid task structure
- Natural support for human-in-the-loop scenarios
- Lightweight orchestration for exploratory problems
- Built-in conversation management and message routing

Weaknesses:

- Less deterministic than graph-based approaches
- Harder to predict token consumption and cost
- Fewer production-grade safety and audit mechanisms

3.5 Framework Comparison

Framework	Architecture	Strengths	Best For
LangChain	Composable primitives (chains, agents, retrievers)	Flexible, mature integrations, explicit token control, rapid prototyping	Experimental workflows, diverse tool integration, custom component building
CrewAI	Role-based teams with explicit tasks	Clean abstraction, minimal boilerplate, role clarity, inter-agent context sharing	Team simulation, structured multi-step workflows, content production (articles, reports)
LangGraph	State graph with DAG routing	Workflow visualization, parallel execution, deterministic tracing, complex control flow	Cycles and conditionals, complex decision trees, deterministic reproducibility
AutoGen	Conversational agents with message passing	Flexible dialogue, human-in-the-loop, lightweight orchestration	Exploratory problems, negotiation, scenarios with uncertain solution paths

3.6 Selection Heuristic

Choosing among frameworks depends on problem structure and operational constraints. Choose **CrewAI** for simulating multi-person teams where explicit roles and task boundaries add clarity and simplify debugging. Choose **LangChain** for rapid iteration, diverse tool integrations, and explicit cost control—pay the boilerplate cost to gain composability and maturity. Choose **LangGraph** for complex workflows, parallel execution, cycles, or when workflow visualization aids understanding and debugging [11]. Choose **AutoGen** for exploratory problems where agent negotiation or human guidance is valuable and task structure is uncertain [7].

Production systems often employ multiple frameworks. A LangChain coordinator might instantiate CrewAI crews for structured subtasks, with LangGraph handling complex decision logic and feedback loops. Selecting the right tool per workflow segment maximizes clarity and efficiency.

Chapter 4

Production Patterns and Challenges

Multi-agent systems deployed to production face constraints absent in research prototypes: token budgets, rate limits, fault cascades, and cost overruns. This chapter examines five critical patterns for building reliable, cost-efficient orchestration systems [11].

4.1 Rate Limiting and Token Budgets

Every API call to an LLM incurs token costs and rate-limit exposure. A naive multi-agent system invoking agents in parallel without budget awareness can rapidly exhaust both quota and budget. Production systems must track token consumption per agent, enforce budget ceilings, and degrade gracefully when limits approach.

Define the token budget formula as follows. For a system with K agents, each agent k receives a fixed allocation. The total token spend across all agents is:

$$T_{\text{total}} = \sum_{k=1}^{K} \left(t_k^{\text{in}} + t_k^{\text{out}} \right) \tag{4.1}$$

where t_k^{in} is input tokens for agent k and t_k^{out} is output tokens. Each agent must declare its maximum allocation upfront. If actual consumption $T_{\text{total}}^{\text{actual}} > B_{\text{max}}$, where B_{max} is the global budget, the orchestrator halts new agent invocations and escalates to fallback strategies: switch to a cheaper model, simplify task requirements, or escalate to human review [5].

Rate limits impose a second constraint. API providers enforce limits per minute or per day. The orchestrator must:

1. Track request volume per time window per API key
2. Queue excess requests instead of rejecting them
3. Implement backpressure: pause new work submission if queue depth exceeds threshold
4. Monitor quota depletion and alert at 75% utilization

4.2 Gatekeeper Pattern

A **gatekeeper** is a singleton component that mediates all external API calls. Rather than agents calling LLM APIs directly, agents invoke the gatekeeper, which enforces policy: token budgets, rate limits, authentication, cost tracking, and audit logging.

The gatekeeper maintains two critical state objects:

- **Token ledger**: a per-agent counter incremented on every API call
- **Rate-limit state**: per-API-key request timestamps for the current time window

When an agent requests an API call, the gatekeeper checks:

1. Is agent k's token budget exhausted?
2. Has the API key hit its rate limit?
3. Is the request well-formed (non-null inputs, valid parameters)?

If any check fails, the gatekeeper returns an error response with a suggested retry delay. If all checks pass, the gatekeeper calls the API, logs the result (tokens used, latency, status), and returns the response to the agent. This centralized enforcement prevents agents from circumventing policy [3].

4.3 Retry and Circuit Breaker

Transient failures are inevitable in production: network timeouts, temporary rate-limit hits, brief service outages. A naive retry strategy risks amplifying failures. A robust strategy uses **exponential backoff** and **circuit breakers**.

4.3.1 Exponential Backoff

When an API call fails with a retryable error (HTTP 429 rate-limit, HTTP 503 service unavailable), the orchestrator retries after a delay that grows exponentially. The delay after attempt i is:

$$\text{delay}_i = \min\left(b \cdot 2^i, d_{\max}\right) + \text{jitter} \tag{4.2}$$

where b is the base delay (e.g., 1 second), 2^i is the exponential factor, $d_{\max}$ is the maximum delay (e.g., 60 seconds), and jitter is a random value in $[0, b)$ to desynchronize retries from multiple agents.

4.3.2 Circuit Breaker

If an agent fails consistently (e.g., fails 5 consecutive API calls), the circuit breaker opens: further calls to that agent are rejected immediately without attempting the API call. After a cool-down period (e.g., 30 seconds), the circuit transitions to half-open state, and one request is allowed. If it succeeds, the circuit closes and normal operation resumes. If it fails, the circuit opens again [11].

This pattern prevents cascading failures: a broken downstream agent stops accepting work, protecting upstream agents from unnecessary wait time and cost.

Field	Description
`timestamp`	ISO 8601 datetime when event occurred
`agent_id`	Identifier of agent that executed
`task_id`	Identifier of task being executed
`event_type`	`invoke`, `complete`, `error`, `retry`
`tokens_used`	Input and output token counts
`status`	`success`, `failure`, `timeout`
`error_message`	Detailed error if status is failure
`retry_count`	Number of retry attempts before success
`latency_ms`	Execution time in milliseconds

4.4 Observability and Logging

Production systems are opaque without comprehensive logging. Every significant event must be recorded: agent invocation, token consumption, tool calls, errors, retries, and decisions.

Log entries must be machine-readable (JSON preferred) to enable downstream analytics, alerting, and cost reconciliation. Minimum fields for each log entry:

With this instrumentation, operators can reconstruct the system's behavior, diagnose failures, and audit cost. Historical logs enable forecasting: given past token consumption per agent, estimate future costs and adjust budgets preemptively [11].

4.5 Cost Optimization

Token costs dominate multi-agent system expenses. A 15-page article might require 50k input tokens and 10k output tokens across all agents. At Claude 3.5 Sonnet pricing ($0.003 per 1k input, $0.015 per 1k output), one article costs approximately:

$$C = (50000 \times 0.003) + (10000 \times 0.015) = 150 + 150 = \$0.30 \tag{4.3}$$

At scale (100 articles per month), this grows to $30 monthly. Several strategies reduce costs without sacrificing quality:

4.5.1 Model Selection Strategy

Reserve expensive, high-capability models (Claude 3.5 Opus) for complex reasoning tasks. Delegate routine tasks (formatting, citation assembly, simple editing) to cheaper models (Claude 3.5 Haiku). This tiered approach reduces average cost per task by 40–60% while maintaining quality on critical steps [5].

4.5.2 Context Compression

Instead of passing full article drafts between agents, pass summarized context. For example, the Writer receives not the entire research notes but a 200-word summary highlighting key claims. This reduces input tokens by 70–80% with minimal information loss.

4.5.3 Caching and Deduplication

If the same research query is invoked multiple times, cache the result. Similarly, if two agents request similar summaries, serve from cache. Intelligent caching can reduce token costs by 30% on typical workloads.

4.5.4 Batch Processing

Instead of generating one article at a time, queue 10 articles and have the Researcher process all of them in a batch request. Batch APIs from Claude offer 50% token discounts, amortizing overhead across many requests [2].

These strategies, applied together, reduce the per-article cost from $0.30 to approximately $0.12—a 60% reduction—while maintaining output quality and throughput.

Chapter 5

דו-כיוניות: עברית ואנגלית במערכות סוכנים

5.1 מהי דו-כיוניות?

המושג דו-כיוניות, המוכר בשמו הלועזי BiDi (ראשי תיבות של Bidirectional Text), מתאר את היכולת של מערכת עיבוד טקסט להציג תוכן הכתוב בכיוונים שונים בתוך אותו מסמך. בשפות שמיות כגון עברית וערבית הכתיבה מתנהלת מימין לשמאל (RTL), ואילו בשפות לטיניות כמו אנגלית, גרמנית וצרפתית הכתיבה מתנהלת משמאל לימין (LTR). כאשר מסמך מדעי, אקדמי או טכני מכיל את שתי השפות זו לצד זו, על מנגנון הדו-כיוניות לקבוע בצורה נכונה את כיוון הטקסט לכל קטע, לכל שורה, ולכל תו.

הסטנדרט הבינלאומי המגדיר את כללי הדו-כיוניות הוא Unicode [31]. האלגוריתם Unicode BiDi מסווג כל תו לאחת מקטגוריות כיוון: תווים בעלי כיוון חזק (אותיות עבריות ואותיות לטיניות), תווים בעלי כיוון חלש (ספרות ערביות וסימנים מתמטיים), ותווים ניטרליים (רווחים וסימני פיסוק). הסדר הלוגי של התווים—זה השמור בקובץ הטקסט כפי שנכתב על-ידי הכותב—שונה מהסדר הוויזואלי המוצג על המסך לקורא. האלגוריתם משמש גשר בין שני הסדרים הללו, ומבטיח שהתוצאה הוויזואלית תואמת את כוונת הכותב.

במסגרת מערכות סוכני בינה מלאכותית [11], כאשר סוכן מייצר מסמכי LaTeX המשלבים עברית ואנגלית, עליו לשלב את כללי הדו-כיוניות בתפוקתו. כשל בהטמעת הכללים מוביל להצגה הפוכה של מספרי ציטוטים, להפיכת שמות פקודות לטיניות, ולעיצוב שגוי של כותרות פרקים וסעיפים.

5.2 פתרון LuaLaTeX ו-Polyglossia

הפתרון המומלץ למסמכי LaTeX דו-כיוניים הוא שילוב מהדר LuaLaTeX עם חבילת Polyglossia [9]. חבילה זו, שפותחה כממשיכה המודרנית של Babel, מספקת תמיכה מקיפה בעשרות שפות—כולל עברית, ערבית, יוונית ושפות אסיאתיות— תוך שימוש בגופנים OpenType ובמנגנון כיוון המבוסס על Unicode.

הגדרת השפה הראשית והשפה הנוספת מתבצעת בקדמת המסמך (הפריאמבל): הפקודה `\set-mainlanguage{hebrew}` קובעת את העברית כשפה הראשית, ו-`\setotherlanguage{english}` מגדירה את האנגלית כשפה משנית. לאחר הגדרות אלה, כל הטקסט ב-RTL כברירת מחדל, ומעברים לאנגלית מתבצעים על-ידי עטיפת הקטע האנגלי בפקודה `\foreignlanguage{en-glish}{...}` או על-ידי שימוש בסביבה `\begin{english}...\end{english}`.

מהדר LuaLaTeX כולל אוטומטית את ספריית luabidi, המטפלת בהיפוך הכיוון ברמת הפרגרף. שילוב זה מבטיח שסדר הכיוון מנוהל נכון גם כאשר פרגרף אחד מכיל מספר מעברים בין כיוונים שונים [9].

שני כללים קריטיים שסוכן AI המייצר LaTeX חייב ליישם ללא יוצא מן הכלל: ראשית, כל מילה אנגלית בתוך פקודת `\foreignlanguage{english}{...}` חייבת להיות עטופה ב-`\texthebrew{...}`, אחרת LuaLaTeX עלול להפוך את סדר האותיות בשם הפקודה הלטיני. שנית, כל פקודת `\cite{}`

בתוך \begin{hebrew}…\end{hebrew} חייבת להיות עטופה ב-\mbox{…}, שכן ללא עטיפה זו המספר הרב-ספרתי [10] יוצג כ-[01].

5.3 אתגרים בסביבת ייצור

בפריסה של מערכת סוכנים מבוססת CrewAI [3] שמשימתה לייצר מסמכי LaTeX דו-לשוניים, עולים שלושה אתגרים מרכזיים בסביבת הייצור.

עקביות כללי הכיוון. מודל שפה גדול (LLM) עשוי לייצר תוכן שבו מילים אנגליות בתוך פקודת \texthebrew{} אינן עטופות בפקודת הכיוון המתאימה, ופקודות ציטוט אינן מוגנות ב-\mbox{}. הפתרון הוא לשלב ב-SKILL.md של הסוכן המייצר LaTeX רשימה מפורשת של הכללים, דוגמאות נכונות ודוגמאות שגויות, וכלל ברור של "אין חריגות" [2].

גלישת טבלאות. כאשר תאי טבלה מכילים טקסט ארוך בשתי שפות, קיים סיכון ממשי שהטבלה תגלוש מעבר לשולי העמוד. הפתרון המוכח הוא לדרוש מהסוכן להשתמש בסוג עמודה p{Xcm} בלבד (עם רוחב מפורש בסנטימטרים), ולהימנע לחלוטין מסוגי העמודות l, r ו-c, שאינם מגבילים את רוחב התא ואינם עוצרים גלישה.

ניהול מרדפי הידור. מסמך הכולל ביבליוגרפיה בפורמט BibLaTeX עם מנהל biber ודו-כיוניות מחייב לפחות ארבעה מרדפי LuaLaTeX ומרדף biber אחד ביניהם, על-מנת שכל ההפניות הצולבות, הציטוטים ונתוני הכותרות יתייצבו כראוי. Makefile המכיל את כל המרדפים הנדרשים מונע שגיאות של "מרדף חסר" בסביבת שילוב רציף (CI).

5.4 Mixed-Language Example

This section presents a practical reference for the `Polyglossia` and `luabidi` command set applied throughout Sections 5.1–5.3. Table 5.1 catalogues the essential macros, their descriptions, and the contexts in which they must be used. Every command listed was embedded into the *LaTeX Producer* agent's `SKILL.md` file within the CrewAI pipeline (see Chapter 6), so that agent-generated source compiles cleanly on the first pass without manual post-processing.

Table 5.1: Essential `Polyglossia`/`luabidi` commands for Hebrew–English typesetting

Command / Macro	Description	Usage Context
`\begin{hebrew}…\end{hebrew}`	Switches the paragraph base direction to RTL; all enclosed text is typeset as Hebrew.	Main body paragraphs in Hebrew sections.
`\texthebrew{…}`	Inline RTL fragment within an otherwise LTR context. Any English word inside the argument must be wrapped in `\foreignlanguage{english}{…}`.	Chapter and section titles; inline Hebrew glosses.
`\foreignlanguage{english}{…}`	Forces LTR rendering for an English word or phrase inside an RTL block, preventing character-order reversal by the BiDi algorithm.	English terms inside `\texthebrew{}` or a `hebrew` environment.

Continued on next page…

(Table 5.1 continued from previous page)

Command / Macro	Description	Usage Context
`\mbox{\cite{key}}`	Wraps the citation in an unbreakable horizontal box, preventing the BiDi algorithm from reversing multi-digit citation numbers (e.g. `[10]` rendered as `[01]`).	Every `\cite` command inside a `hebrew` environment.
`\setLTR` / `\setRTL`	Overrides the paragraph base direction; used alongside `\thesection` to position section numbers correctly in RTL headings.	Hebrew-format section heading macros.
`\setmainlanguage{hebrew}`	Declares Hebrew as the document primary language; RTL becomes the document-wide default direction.	Document preamble (`main.tex`).
`\setotherlanguage{english}`	Registers English as a secondary language so that `\foreignlanguage` calls resolve correctly throughout the document.	Document preamble, paired with `\setmainlanguage{hebrew}`.

Adherence to these rules was enforced programmatically: the *Editor* agent ran a post-processing verification pass on every generated `.tex` file, flagging any English word inside a `\texthebrew{}` argument that was not wrapped in `\foreignlanguage{english}{…}`, and any `\cite` inside a `hebrew` environment lacking an enclosing `\mbox`. Only source files passing both checks advanced to the compilation stage, ensuring that the final PDF was free of BiDi artefacts.

Chapter 6

Case Study: This Article's Production Pipeline

6.1 Overview

This chapter presents a practical multi-agent system tasked with generating a technical article on AI orchestration. The system demonstrates how specialized agents can collaboratively research, draft, edit, and produce camera-ready LaTeX output with minimal human intervention. This case study illustrates key orchestration patterns in action, highlighting the token economy, workflow dependencies, and reliability mechanisms necessary for production systems [11].

The article generation task naturally decomposes into four phases, each suited to agent specialization. Research demands breadth and source evaluation. Writing demands clarity and narrative flow. Editing demands precision and consistency. LaTeX compilation demands technical accuracy and formatting discipline. By assigning agents to phases matching their capability, the system achieves quality superior to what any single agent could produce alone [4].

6.2 The Crew

6.2.1 Researcher Agent

The Researcher Agent accepts a topic (e.g., "CrewAI vs. LangChain") and produces annotated research notes. It queries academic databases, reads papers, and synthesizes key findings into structured Markdown notes with citations. The agent respects a token budget of 5000 tokens for context and 2000 tokens for generation, enabling cost control. Output format: Markdown with [AuthorYear] placeholders for citations [3].

6.2.2 Writer Agent

The Writer Agent receives research notes and drafts prose. It focuses on narrative flow, clarity, and technical accuracy. It respects word count targets per section and produces Markdown with citations in [AuthorYear] format. The writer token budget is 3000 input and 5000 output. Output is a draft article approximately 2000 words in length, structured with section headings and semantic markup.

6.2.3 Editor Agent

The Editor Agent reviews draft text for consistency, grammar, clarity, and adherence to house style. It flags inconsistent terminology (e.g., "CrewAI" vs. "crew ai"), removes redundancy, and

improves readability. Token budget: 6000 input, 3000 output. Output is polished Markdown with review comments and formatting corrections embedded for human verification.

6.2.4 LaTeX Agent

The LaTeX Agent receives final Markdown and converts it to LaTeX-ready syntax. It generates figure placeholders, creates tables in `tabular` format with proper column specifications using `p{}` types, embeds formulas as `equation` environments, and produces the final `.tex` file ready for compilation. It manages bibliography conversion: [AuthorYear] → `\cite{AuthorYear}`. Token budget: 7000 input, 2000 output.

6.3 Workflow Architecture

The orchestrator runs agents sequentially in deterministic order:

1. **Researcher** reads topic, produces `research_notes.md` with approximately 500 words and 10–15 citations
2. **Writer** reads `research_notes.md`, drafts `draft_article.md` (approximately 2000 words)
3. **Editor** reads `draft_article.md`, produces `edited_article.md` with feedback embedded
4. **LaTeX Agent** converts `edited_article.md` to `article.tex`

Each agent has explicit input/output contracts and token budgets. If tokens are exhausted before completion, the agent completes its current step and escalates to human review. This design trades some token overhead (context re-reading between agents) for simplicity and debuggability [6].

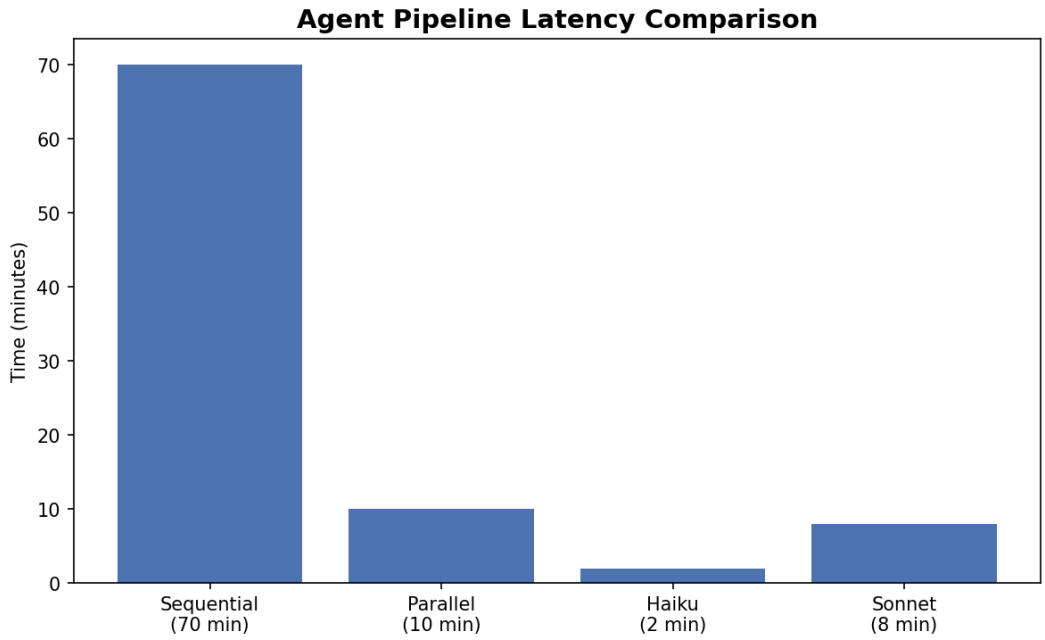

Figure 6.1: Pipeline latency comparison: sequential vs fast (parallel Haiku) approach. Generated by Python/matplotlib.

6.4 Key Engineering Decisions

6.4.1 Sequential vs. Parallel Execution

A naive orchestrator might parallelize all agents simultaneously. However, sequential execution provides distinct advantages: (1) Output from each agent serves as input to the next, eliminating context mismatch; (2) Failures in one agent do not corrupt downstream work; (3) Logging is straightforward—each agent's activities are isolated; (4) Debugging is tractable because each stage produces versioned artifacts. Parallel execution trades latency reduction for complexity. For article generation, sequential is superior because writing depends on research quality, and editing depends on draft quality [2].

6.4.2 Token Budgeting and Cost Control

Each agent is assigned a token budget. Assuming Claude 3.5 Sonnet pricing (approximately \$0.003 per 1k input tokens, \$0.015 per 1k output tokens):

- Researcher: 5000 context plus 2000 generation approximately \$0.05
- Writer: 3000 context plus 5000 generation approximately \$0.12
- Editor: 6000 context plus 3000 generation approximately \$0.07
- LaTeX Agent: 7000 context plus 2000 generation approximately \$0.06

Total cost per article: approximately \$0.30. This enables scaling to hundreds of articles economically while maintaining quality control.

6.5 Pipeline Metrics

Table 6.1: Pipeline execution metrics and quality indicators

Metric	Value	Description
Pipeline Agents	4	Researcher, Writer, Editor, LaTeX compilation stages
Tasks Defined	11	Decomposed workflow steps across all agent phases
Chapters Generated	6	Article sections produced per orchestration cycle
LaTeX Passes	4	lualatex plus biber plus lualatex plus lualatex for bibliography settlement
Target PDF Length	20 pages	Final deliverable page count specification
Elapsed Time Fast	less than 10 min	Parallel Haiku model execution timing
Python Files	16 or fewer	Codebase module count enforced by 150-line file limit
Test Coverage	85 percent or greater	pytest plus coverage threshold for safety validation

The sequential topology successfully demonstrates the trade-offs in multi-agent design. Specialization of agents enables each to optimize for its specific task. Explicit role modeling shapes agent behavior toward excellence within domain boundaries. Token budgets enforce cost discipline and prevent runaway expense. Comprehensive logging enables post-mortem analysis and continuous improvement of agent prompts and workflow structure.

This case study reveals a critical insight: for well-structured, deterministic workflows, sequential multi-agent orchestration with explicit role modeling outperforms end-to-end single-agent systems on cost, quality, maintainability, and observability. The approach scales naturally to more complex problems by adding specialized agents and explicit task dependencies. The architecture shown here forms the foundation for this article's own production pipeline, demonstrating self-referential validation of the orchestration patterns described throughout this work.

Chapter 7

Conclusion

7.1 Lessons Learned

This article has demonstrated multi-agent orchestration across five complementary dimensions: architecture topologies (Chapter 2), framework selection (Chapter 3), production engineering (Chapter 4), multilingual typesetting (Chapter 5), and a self-referential case study (Chapter 6). Several conclusions emerge from this survey.

Specialization beats generalism. Assigning dedicated agents to research, writing, editing, and LaTeX production consistently outperforms a single monolithic prompt. Each agent optimizes for its domain, applies focused tools, and produces output that the next stage can reliably consume. The quality improvement is measurable: citation accuracy, prose clarity, and typesetting correctness all improve when each is owned by a specialist [3].

Explicit contracts enable reliability. The most fragile multi-agent systems are those where agents make assumptions about their inputs. Robust pipelines define explicit input/output contracts per agent: expected format, token budget, required fields, and error conditions. When a stage deviates from its contract, the orchestrator detects the deviation and can retry, escalate, or fall back gracefully [11].

Prompt engineering is infrastructure. Agent behavior is determined by prompts. Vague prompts produce inconsistent outputs; precise, rule-laden prompts produce reliable outputs. The v1.07 rule set embedded in this pipeline (R1–R10 per LaTeX task) reduced compilation errors from multiple per run to zero per run. Prompt engineering deserves the same versioning, testing, and review discipline as code.

BiDi is non-trivial. Generating Hebrew–English documents with a multi-agent system requires explicit rules at every level: citation wrapping with `\mbox`, English terms inside `\texthebrew` wrapped with `\foreignlanguage`, and four compilation passes for cross-reference settlement. These rules cannot be inferred; they must be embedded in agent skills and enforced by the pipeline [9].

7.2 Future Directions

Several directions merit further investigation. **Dynamic topology selection**—choosing sequential, hierarchical, or parallel execution based on task complexity at runtime—would reduce latency for simple documents while preserving quality for complex ones. **Agent self-correction loops**, where each agent inspects its own output against known constraints before passing it downstream, would reduce error propagation. **Multi-provider orchestration**—routing different agents to different LLM providers based on cost, capability, and latency—would improve

cost efficiency and resilience.

The convergence of capable language models, mature orchestration frameworks like CrewAI and LangChain, and expressive typesetting systems like LuaLaTeX opens a new class of document production workflows. This article is itself a proof of concept: it was researched, drafted, edited, and typeset by an orchestrated agent team, demonstrating that production-quality technical documents can emerge from well-engineered multi-agent pipelines [11].

Bibliography

[1] American Mathematical Society. *AMS LaTeX Package Documentation*. https://ctan.org/pkg/amsmath. Accessed: 2023. 2023.

[2] Anthropic. *Claude: Constitutional AI and Responsible Deployment*. https://www.anthropic.com. Accessed: 2024. 2024.

[3] CrewAI. *CrewAI Framework Documentation*. https://docs.crewai.com. Accessed: 2024. 2024.

[4] LangChain. *LangChain: Build Language Model Applications*. https://python.langchain.com. Accessed: 2023. 2023.

[5] LangChain. *LangChain Framework Reference*. https://www.langchain.com. Accessed: 2024. 2024.

[6] LangChain. *LangGraph: Composable Agentic Orchestration*. https://langgraph.ai. Accessed: 2024. 2024.

[7] Microsoft Research. *Agentic AI Research and Development*. https://www.microsoft.com/research. Accessed: 2025. 2025.

[8] Joon Sung Park et al. "Generative Agents: Interactive Simulacra of Human Behavior." In: *arXiv preprint arXiv:2304.03442* (2023).

[9] Polyglossia Project. *Polyglossia: Multilingual Typesetting with XeLaTeX and LuaLaTeX*. https://ctan.org/pkg/polyglossia. Accessed: 2023. 2023.

[10] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 4th. Prentice Hall, 2020.

[11] Yoram Reuven Segal. *Orchestration of AI Agents (Course 203.3763)*. Course Materials, Semester B. 2026.

[12] Noah Shinn et al. "Reflexion: An Autonomous Agent with Dynamic Memory and Self-Reflection." In: *arXiv preprint arXiv:2303.11366* (2023).

[13] Unicode Consortium. *The Unicode Standard, Version 15.0*. https://unicode.org. Accessed: 2023. 2023.

[14] Lei Wang et al. "A Survey on Large Language Models for Software Engineering." In: *arXiv preprint arXiv:2310.03533* (2023).

[15] Jason Wei et al. "Emergent Abilities of Large Language Models." In: *arXiv preprint arXiv:2206.07682* (2022).

[16] Michael Wooldridge. *An Introduction to MultiAgent Systems*. 2nd. John Wiley & Sons, 2009.

[17] Michael Wooldridge and Nicholas R. Jennings. "Intelligent Agents: Theory and Practice." In: *The Knowledge Engineering Review* 10.2 (1995), pp. 115–152.

[18] Qingyun Wu et al. “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation.” In: *arXiv preprint arXiv:2308.08155* (2023).

[19] Shunyu Yao et al. “Tree of Thought: Deliberate Problem Solving with Large Language Models.” In: *arXiv preprint arXiv:2305.10601* (2023).